

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping:**
A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage:**
In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in

storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.20;
```

```
/**
```

```

* @title Ballot
* @dev Implements voting process along with vote delegation
*/
contract Ballot {

    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    struct Proposal {
        string name;    // ✅ store full string
        uint voteCount;
    }

    address public chairperson;

    mapping(address => Voter) public voters;

    Proposal[] public proposals;

    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(
                Proposal({
                    name: proposalNames[i],
                    voteCount: 0
                })
            );
        }
    }

    function giveRightToVote(address voter) external {
        require(msg.sender == chairperson, "Only chairperson can give right to vote.");
        require(!voters[voter].voted, "The voter already voted.");
    }
}

```

```

    require(voters[voter].weight == 0, "Voter already has the right to vote.");

    voters[voter].weight = 1;
}

function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right to vote.");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");
    require(proposal < proposals.length, "Invalid proposal.");

    sender.voted = true;
    sender.vote = proposal;
}

```

```

        proposals[proposal].voteCount += sender.weight;
    }

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() external view returns (string memory) {
    return proposals[winningProposal()].name;
}
}

```

Implementation:

Deployed

The screenshot displays the Remix IDE interface during the deployment of a Solidity contract. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel shows the contract 'Ballot - contracts/3_Ballot.sol' with an EVM version of 'osaka'. The 'Deploy' button is highlighted, and the 'At Address' button is also visible. Below this, it indicates 'Transactions recorded: 26' and 'Deployed Contracts: 2'. The bottom status bar shows 'BALLOT AT 0XD4f...2CBEE (MEV)' and a balance of '0 ETH'.

On the right, the 'Filter with transaction hash or address' search bar is active. The 'decoded input' section shows the following JSON structure:

```

{
  "string[] proposalNames": [
    "Rujuta",
    "Madhura",
    "Shravani"
  ]
}

```

The 'decoded output' section shows a single dash '-', indicating no output data. The 'logs' and 'raw logs' sections are empty. The bottom status bar indicates 'creation of Ballot pending...'.

Chairperson gave right to vote to 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

Transactions recorded 27 i >

Deployed Contracts 2

BALLOT AT 0XD4F...2CBEE (MEM)

Balance: 0 ETH

delegate address to

giveRightToVote 0xAb8483F64d9C6d1EcF9b8

vote uint256 proposal

chairperson

proposals uint256

voters address

transact to Ballot.giveRightToVote pending ...

[vm] from: 0x5B3...eddC4
to: Ballot.giveRightToVote(address) 0xD4F...2cbee value: 0 wei
data: 0x9e7...35cb2 logs: 0 hash: 0x6ce...e516e

Debug

status 1 Transaction mined and execution succeed

transaction hash 0x6ce495352795ce06fd7823dfb3e75a632b48ef4bfbdb7e516e

block hash 0x09767ffc162a183315309b3e79d9e42c4727b5ada101cde6886ee441c7f05f59

block number 27

from 0x5B38Da6a701c568545dCfc8B75F56beddC4

to Ballot.giveRightToVote(address)
0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

transaction cost 48642 gas

execution cost 27210 gas

output 0x

Voted to person at index 0

DEPLOY & RUN
TRANSACTIONS

Deployed Contracts 2

BALLOT AT 0XD4F...2CBEE (MEM)

Balance: 0 ETH

delegate address to

giveRightToVote 0xAb8483F64d9C6d1EcF9b8

vote 0

chairperson

proposals uint256

voters address

winnerName

winningProposal

Low level interactions i

0 Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
value: 0 wei data: 0x012...00000 logs: 0 hash: 0xbc7...55e7e

Debug

status 1 Transaction mined and execution succeed

transaction hash 0xbc74692df16cb13c56f76d2a73a9d14f5bd6c459751a99451e4f5024de155e7e

block hash 0xab989142d706cfe9985d0a4d968b27dfe591ba010d8cfc029e3a084bc6634232

block number 28

from 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

to Ballot.vote(uint256) 0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

transaction cost 73115 gas

execution cost 51923 gas

output 0x

decoded input {
"uint256 proposal": "0"
}

decoded output {}

logs []

Winner name

[illegible]

Chairperson gave right to vote to 0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db

DEPLOY & RUN TRANSACTIONS

▼ BALLOT AT 0XD4F...2CBEE (MEN) 📄 📌 ✕

Balance: 0 ETH

delegate

▼

giveRightToVote

▼

vote

▼

chairperson

proposals

▼

voters

▼

winnerName

0: string: Rujuta

winningProposal

0 ○ Listen on all transactions

Filter with transaction hash or ad...

🔄 ⚙️ ✕

✓

[vm] from: 0x5B3...eddC4
to: Ballot.giveRightToVote(address) 0xD4F...2cbee value: 0 wei
data: 0x9e7...c02db logs: 0 hash: 0x4e4...774ea

Debug

status	1 Transaction mined and execution succeed
transaction hash	0x4e4a9075b24dfb3cadb44eb43911c717e85d545f4c78a22b0c897de13c9774ea 📄
block hash	0x1470f032e9286b23fed1fa5a0f5fe0cbald44427fb6c8ceb7108d6e9c07fcab 📄
block number	29 📄
from	0x5B38Da6a701c568545dCfc803fc8875f56beddC4 📄
to	Ballot.giveRightToVote(address) 0xD4Fc541236927E2EAf8F27606bD7309C1Fc2cbee 📄
transaction cost	48642 gas 📄
execution cost	27210 gas 📄
output	0x 📄
decoded input	{ "address voter": "0x4820993Bc481177ec7E8f571ceCaE8A9e22C02db" }

Gave vote to person at index 1

DEPLOY & RUN
TRANSACTIONS

▼ BALLOT AT 0XD4F...2CBEE (MEV)

Balance: 0 ETH

delegate

address to

giveRightToVote

0x4B209938c481177ec7E8f5

vote

1

chairperson

proposals

uint256

voters

address

winnerName

0: string: Rujuta

winningProposal

Low level interactions

CALLDATA

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

transact to Ballot.vote pending ...

✓ [vm] from: 0x5B3...eddC4 to: Ballot.vote(uint256) 0xD4F...2cbee value: 0 wei data: 0x012...0001 logs: 0 hash: 0x4d6...d164a

Debug

status

1 Transaction mined and execution succeed

transaction hash

0x4d621d2881c22a17ab6a29a360c0837fdad96f7eb1b21e354c4316af5b9d164a

block hash

0x235420947c64ac48179e799be2779a663f3df4eab5af10a84dd0dd57c9b08e7c

block number

30

from

0x5B380a6a701c568545dcfc803fc8875f56beddC4

to

Ballot.vote(uint256) 0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

transaction cost

93027 gas

execution cost

71823 gas

output

0x

decoded input

{
 "uint256 proposal": "1"
}

decoded output

{}

Chairman gave right to vote to 0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB

Deployed Contracts 2

✓ BALLOT AT 0XD4F...2CBEE (MEV)

Balance: 0 ETH

delegate

address to

giveRightToVote

0x78731D3Ca6b7E34aC0F82

vote

1

chairperson

proposals

uint256

voters

address

winnerName

0: string: Rujuta

winningProposal

transact to Ballot.giveRightToVote pending ...

✓ [vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xD4F...2cbee value: 0 wei data: 0x9e7...cabab logs: 0 hash: 0x1c6...38563

Debug

status

1 Transaction mined and execution succeed

transaction hash

0x1c66389537780983d242e4df719fdbd7a71911684f64ba33fa5a62dd31f38563

block hash

0xf37a77ace00680d51c7f56dea83b931012eb5d025f8037fecca22bdb604014bd

block number

32

from

0x5B380a6a701c568545dcfc803fc8875f56beddC4

to

Ballot.giveRightToVote(address)
0xD4Fc541236927E2EAF8F27606b07309C1Fc2cbee

transaction cost

48642 gas

execution cost

27210 gas

output

0x

delegate address to

giveRightToVote 0x78731D3Ca6b7E34aC0F82

vote 1

chairperson

proposals uint256

voters address

winnerName

0: string: Rujuta

winningProposal

```
logs []
raw logs []
transact to Ballot.vote pending ...

[vm] from: 0x787...cabaB to: Ballot.vote(uint256) 0xD4F...2cbec
value: 0 wei data: 0x012...00001 logs: 0 hash: 0x94f...81b20

status 1 Transaction mined and execution succeed
transaction hash 0x94fb2c75b57e29dcff2e911d82096d0b9bcad778d8174609b7ec2cf581481b20
block hash 0x34e79f4ebe4d2b7fba94e1c3393bf9db3fd379d2f4684d39f0b46e22d51b9c0b
block number 33
from 0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB
to Ballot.vote(uint256) 0xD4Fc541236927E2EAf8F27606bd7309C1Fc2cbec
```

[illegible]

In this experiment, a Voting/Ballot Smart Contract was successfully deployed and tested. The chairperson granted voting rights, voters cast their votes, and the contract correctly determined the winner based on vote count. The use of `require` statements ensured security and prevented invalid actions such as double voting. This experiment demonstrated how blockchain technology enables secure, transparent, and reliable voting using smart contracts.